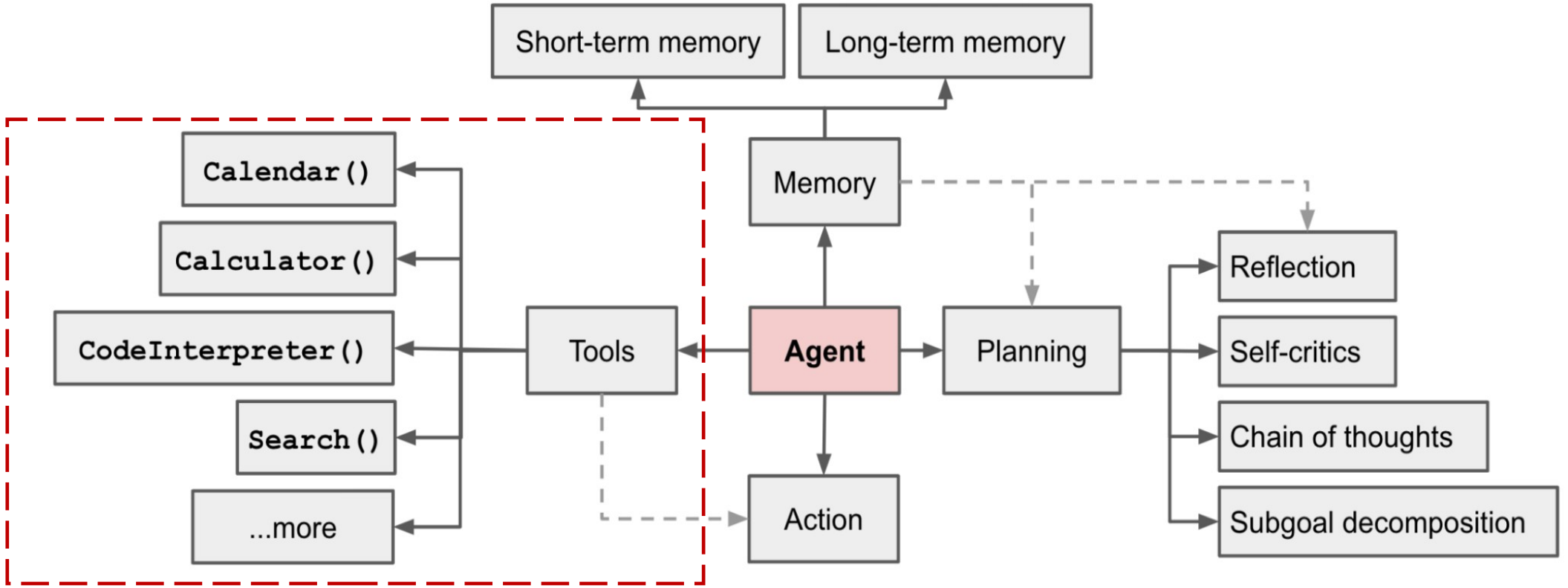
# 通用人工智能创新实践课 - 智能体的工具使用

# 工具和智能体

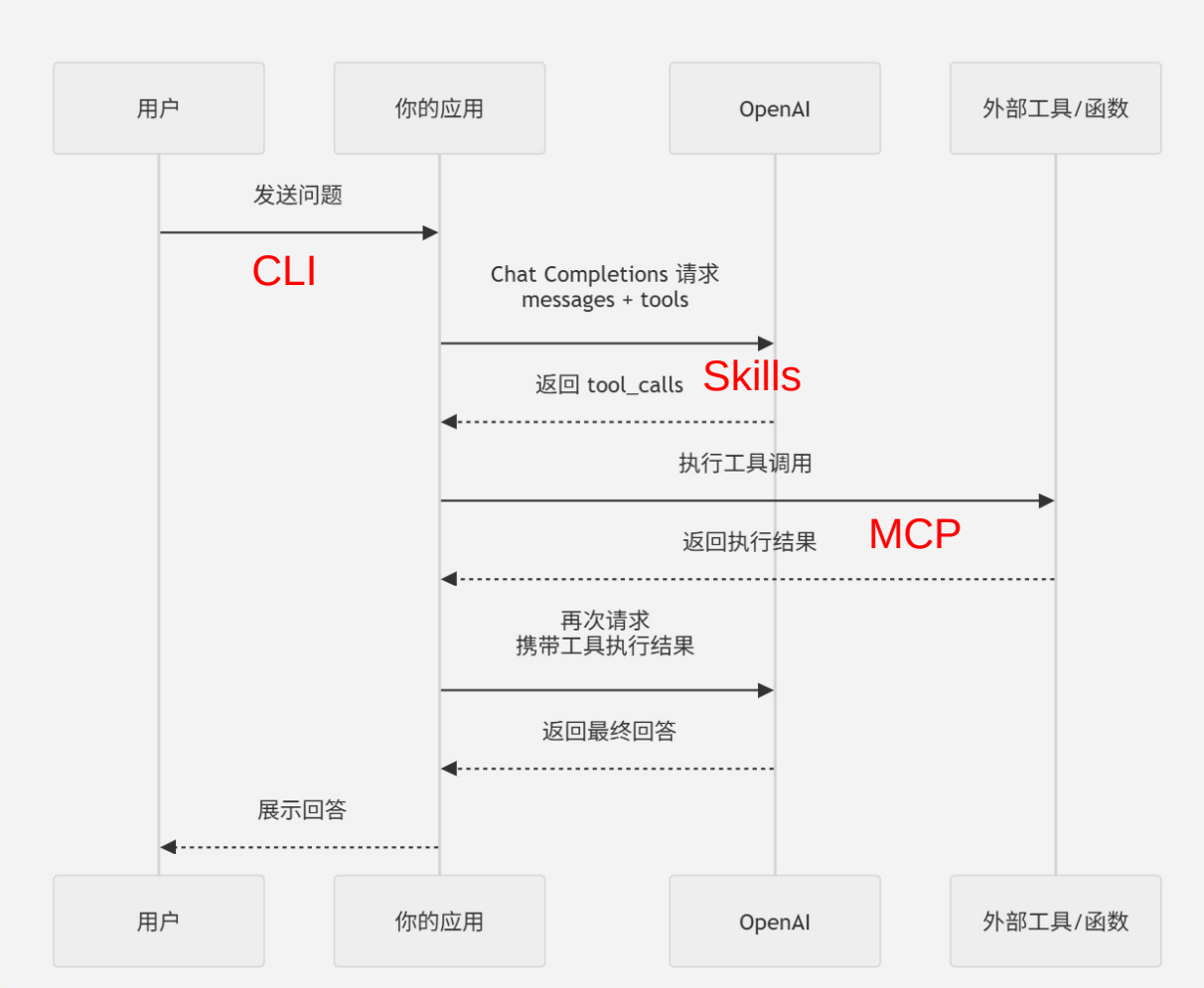



Tool MCP Skills CLI

# 工具技能进化


From Human Skills to Agent Skills
Human Ingenuity Builds the Foundation.
Intelligent Agents Take It Further.
1
~200k BCE – ~10k BCE
Ancient Survival & Tool Skills
2
~3000 BCE – 1600 CE
Knowledge & Engineering Skills
3
~1760s – 1900s
Industrial Skills
4
1970 – 2023
Digital Skills
5
2023 – 2024
Tool Skills
6
2024 – Present
Agent Skills (Next-Gen AI)
Planning
Memory
Tool Use
Collaboration
Fire Making
Tool Use
Hunting & Gathering
Early Farming
Writing
Mathematics
Engineering
Mechanization
Factories
Mass Production
Programming
Internet
Data Processing
Robotics & Automation
Robotics
Automation
AI Systems
Human-AI Interaction
Survive
Master Nature
Create
Build Civilization
Industrialize
Scale with Systems
Digitize
Expand Knowledge
Collaborate & Evolve
Partner with Intelligent Agents

# Tool - 基本方法




```
"tools": [
  {
    "type": "function",
    "function": {
      "name": "get_current_weather",
      "description": "Get the current weather in a given location",
      "parameters": {
        "type": "object",
        "properties": {
          "location": {
            "type": "string",
            "description": "The city and state, e.g. San Francisco, CA"
          },
          "unit": {
            "type": "string",
            "enum": ["celsius", "fahrenheit"]
          }
        },
        "required": ["location"]
      }
    }
  }
],
```

**工具定义**

**工具调用（Tool Calling）= 扩展 LLM 的行动能力**

- **工具定义**：向模型描述可用工具、功能和参数
- **工具选择**：LLM 根据用户意图判断是否需要调用工具
- **结果回传**：应用执行工具，并将结果返回给 LLM

**LLM 从“只会生成内容”→“能够调用外部能力”**
可以进一步连接 **搜索、数据库、业务 API、代码执行等外部系统**，这也是构建 **AI Agent** 的核心能力之一。

# MCP - 基本概念

## 基本概念

- MCP （Model Context Protocol）由 Anthropic 于 2024 年 11 月发起并开源

已发展为开源标准，标准化 LLM 与外部工具、数据源和服务之间的连接方式。

- 官方文档： https://modelcontextprotocol.io/
- 开发仓库： https://github.com/modelcontextprotocol

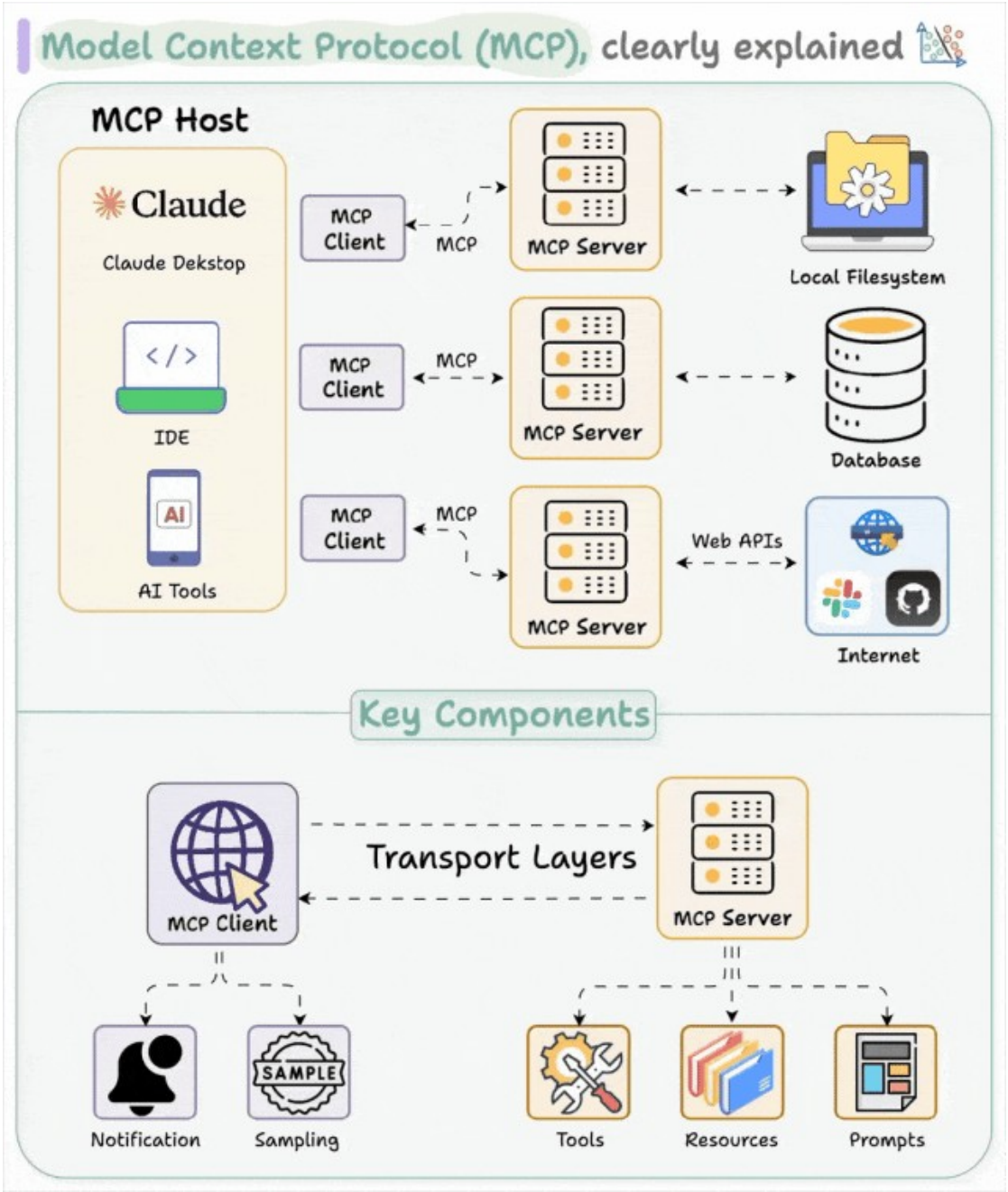
## 设计目标

- 面向开发者：降低智能体开发时间和复杂度
- 面向 Agent：丰富扩展能力，提供权限管理

## H-S-C 架构

- MCP Host：协调和管理一个或多个 MCP 客户端的 AI 应用
- MCP Server：为 MCP 客户端提供数据上下文的程序
- MCP Client：负责维护与 MCP 服务器的连接，并从该 MCP 服务器获取上下文供 MCP 主机使用






MCP 系统拓扑

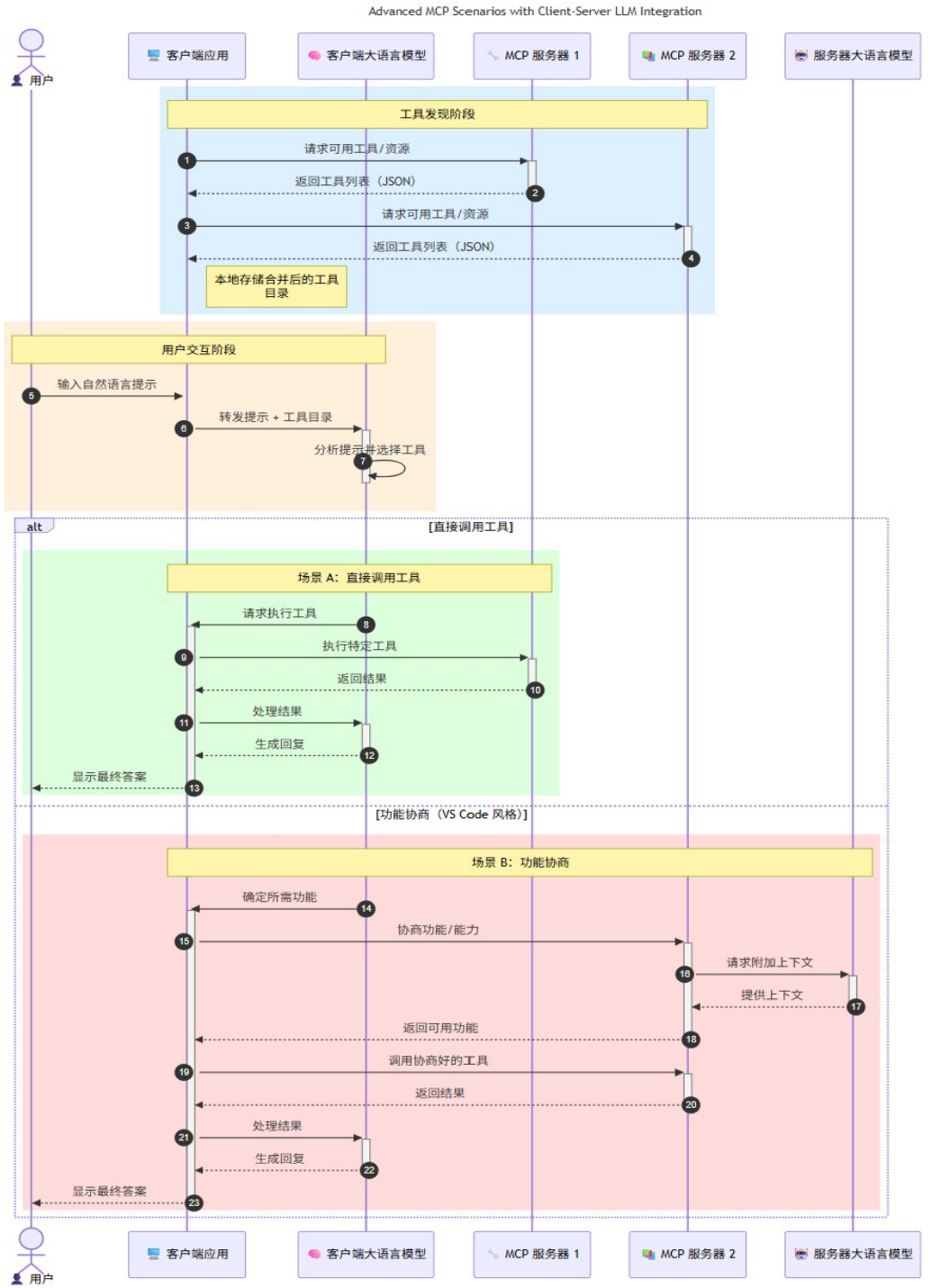
# MCP - 协议

## 数据层

- 基础协议。基于 JSON-RPC 2.0 协议通信，定义消息结构和语义。
- 发现。允许客户端通过请求查询服务器支持的协议版本、能力和身份。
- 请求执行。工具执行要求，进度和结果。
- 订阅通知。客户端选择加入订阅主题；服务器事件驱动，尽力而为。

## 传输层

- 管理客户端与服务器之间的通信通道和认证。
- 两种传输机制
  - Stdio 传输。标准输入 / 输出流实现同一机器上本地进程间通信，无网络开销。
  - Streamable HTTP 。 HTTP POST 处理客户端到服务器消息。
- 认证和授权， HTTP 机制基于 OAuth2.x 协议。支持第三方授权模式。
- 生命周期管理，版本协商 -> 能力协商 -> 操作 -> 关闭。支持超时和优雅关闭



# MCP - Server

## 对外提供基本功能

- Prompts （提示）：预定义模板或指令，用于指导语言模型交互
- Resources （资源）：结构化数据或内容，为模型提供额外的上下文
- Tools （工具）：可执行功能，允许模型执行操作或检索信息

## 部署模式：local / remote

## Prompt

- 接口： prompts/list, prompts/get
- 多模态提示，图片 / 音频需要进行 Base64 编码， MIME 格式

## Resources

- 接口： resources/list, resources/read, subscriptions/listen

## Tools

- 接口： tools/list, tools/call

```python
from typing import Any

import httpx2
from mcp.server import MCPServer
from pydantic import BaseModel, Field, RootModel

# Initialize the MCP server
mcp = MCPServer("weather")

# Constants
NWS_API_BASE = "https://api.weather.gov"
USER_AGENT = "weather-app/1.0"

class Alert(BaseModel): ...
class Alerts(RootModel[list[Alert]]): ...
class Period(BaseModel): ...
class Forecast(BaseModel): ...
async def make_nws_request(url: str) -> dict[str, Any] | None: ...


@mcp.tool()
async def get_alerts(state: str) -> Alerts:
    """Get weather alerts for a US state.

    Args:
        state: Two-letter US state code (e.g. CA, NY)
    """
    url = f"{NWS_API_BASE}/alerts/active/area/{state.upper()}"
    data = await make_nws_request(url)

    if not data or "features" not in data:
        raise ValueError(f"Unable to fetch alerts for {state.upper()}.")

    # An empty result is an empty array, not an error.
    #
    # `or` rather than a `get` default: NWS sends these fields as explicit
    # nulls rather than omitting them, and a default only covers a missing key.
    return Alerts( ... )


@mcp.tool()
async def get_forecast(latitude: float, longitude: float) -> Forecast: ...


def main():
    # Initialize and run the server
    mcp.run(transport="stdio")


if __name__ == "__main__":
    main()
```

weather.py

# MCP - Client

## 基本流程

- MCP Host 创建 MCP Client
- MCP Client 与 Server 建立连接，能力发现
- MCP Host 将 Server 提供的 Prompt/resource/tool 连同 raw query 一起交给 LLM 处理

## 复杂功能

- 渐进式工具发现，目录 -> 描述 -> 执行细节
- 动态服务器管理， server/discover
- 程序化，组合工具调用
- 缓存，订阅和 TTL

```python
from mcp import Client, StdioServerParameters
from mcp.client.stdio import stdio_client
from mcp_types import TextContent

class MCPClient:
    def __init__(self): ...

    @property
    def anthropic(self) -> Anthropic: ...

    async def connect_to_server(self, server_script_path: str):
        """ ...
        """

        # "auto" probes server/discover, falling back to the 2025-11-25 handshake.
        self.client = await self.exit_stack.enter_async_context(
            Client(stdio_client(server_params), mode="auto")
        )

        # List available tools
        response = await self.client.list_tools()
        tools = response.tools
        print(f"\nConnected over protocol {self.client.protocol_version} with tools:", [tool.name for tool in tools])

    async def process_query(self, query: str) -> str:
        """Process a query using Claude and available tools"""
        messages = [{"role": "user", "content": query}]

        tools_response = await self.client.list_tools()
        available_tools = [ ... ]

        final_text = []

        response = self.anthropic.messages.create(
            model=ANTHROPIC_MODEL, max_tokens=MAX_TOKENS, messages=messages, tools=available_tools
        )

        for _ in range(MAX_TOOL_TURNS): ...

        final_text.append(f"[Stopped after {MAX_TOOL_TURNS} tool-use turns]")
        return "\n".join(final_text)

    async def chat_loop(self): ...

    async def cleanup(self):
        """Clean up resources"""
        await self.exit_stack.aclose()


async def main():
    if len(sys.argv) < 2:
        print("Usage: python client.py <path_to_server_script>")
        sys.exit(1)

    client = MCPClient()
    try: ...
    finally:
        await client.cleanup()


if __name__ == "__main__":
    import sys

    asyncio.run(main())
```

client.py

# MCP 生态

| 10,000+ | 9,700万+ | 90,000+ | 16,000 |
| --- | --- | --- | --- |
| 公开 MCP Server 数量 | SDK 月下载量 | GitHub Stars | 相关 GitHub 仓库 |

## 核心推动者与治理

协议由 **Anthropic** 于 2024 年 11 月发起并开源，后捐赠给 **Linux 基金会旗下 Agentic AI Foundation（AAIF）** 进行中立治理，从单一厂商协议升级为开放生态公共设施。

**全球科技巨头**

OpenAI · Google · Microsoft · Amazon

**中国主要厂商**

阿里巴巴 · 百炼 · 腾讯云 · 百度 · 字节跳动 · 蚂蚁集团 · OceanBase

## 云服务平台

主流云厂商提供 MCP 的**部署与运行环境**，Server 可托管运行。

AWS · Azure · GCP · Cloudflare

全托管部署 · 远程调用 · 企业级运维

## IDE · 编程助手 · 应用客户端

开发者**在编码环境直接连接外部工具**；AI 应用让终端用户直接受益。

Cursor · Windsurf · Claude Code · Qoder · GitHub Copilot · VS Code · Claude Desktop · ChatGPT · Gemini · Microsoft Copilot

蓝色为 AI 应用与客户端，青色为开发工具

## AI 应用开发框架

主流框架**原生支持 MCP**，Agent 接入工具生态的开发流程大幅简化。

LangChain · LlamaIndex · AutoGen · Bee Agent · mcp-agent · OpenAI Agents SDK

多语言 SDK：Kotlin / C++ / Go 等

## 著名开源项目

社区贡献集中体现：**Arcade 框架支撑 7500+ 预构建工具**，工具与文档类 Server 最活跃。

n8n 187K★ · Context7 55K★ · Chrome DevTools 39K★ · Playwright 32K★ · GitHub 30K★ · FastMCP 25K★ · Serena 24K★

官方 servers 近 90K★ · awesome-mcp-servers 87K★

**覆盖 模型厂商 × 云服务商 × 开发工具 × 应用框架 × 开源社区 的完整技术生态**

# Skills - 基本概念

## 基本概念

- 最早由 Anthropic 创建，但已经演化为一个开放标准
- 定义一种标准化的程序性知识封装格式
- 跨平台、可迁移、可组合

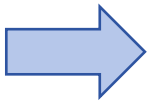
## 解决问题

- 沉淀领域专业知识
- 定义可重复工作流
- 新能力扩展

**单一用途 Agent -> 通用 Agent + Skills**

## 最小形态

```
my-skill/
└── SKILL.md
```



```
---
name: my-skill                    # ← 上半部分：元数据
description: >-                   #    AI 靠这里决定要不要激活这个技能
  当用户需要做某件事时，使用这个技能。
---

下半部分：操作指令                  # ← AI 激活技能后才会读到这里
按照以下步骤执行...
```

## 完整形态

```
skill-name/
├── SKILL.md                  # [必需] 入口文件：frontmatter + body
├── agents/
│   └── openai.yaml           # [推荐] 技能的"名片"
├── scripts/                  # [可选] 可执行脚本
├── references/               # [可选] 参考文档
└── assets/                   # [可选] 产出物模板
```

# Skills - 渐进式披露

## 目标

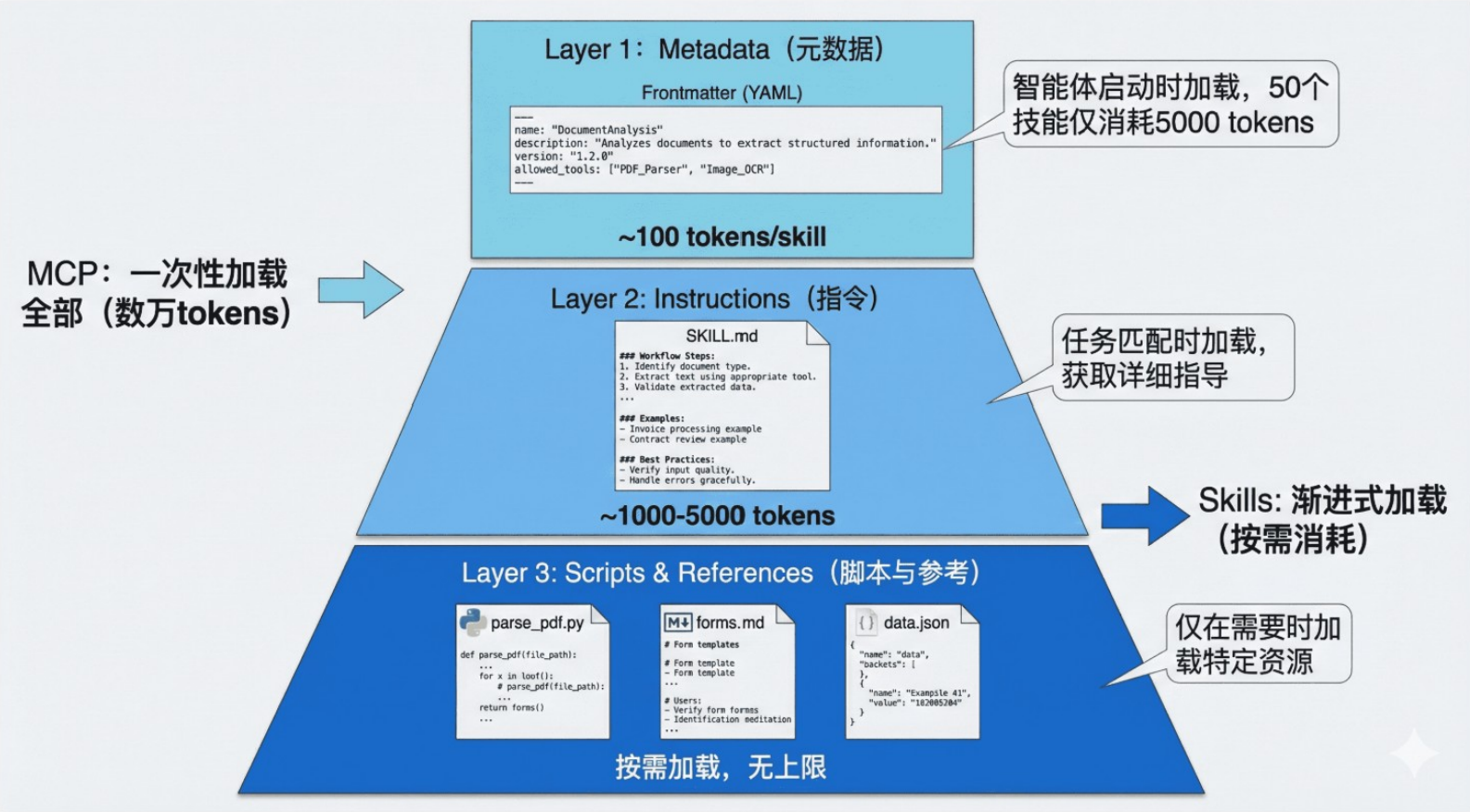
- 解决上下文爆炸问题
- 提升效率、节约成本

## 基本要素

- Layer1： 元数据， FrontMatter
- Layer2 ： 技能主体，指令、注意事项
- Layer3： 附加资源，脚本，资产，文件

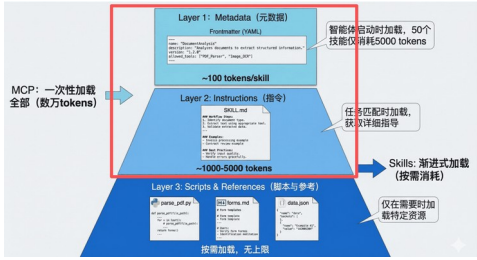
## 工作流程 - 分层懒加载

- Skills 从文件系统加载时，仅 name 和 description 进入上下文窗口
- LLM 根据描述判断何时触发某个 Skill
- 触发后加载 SKILL.md 完整内容
- 如果指令中引用了额外文件或脚本，按需逐步加载




渐进式披露示意图

# Skills - 元信息和指令




## 元信息

- 位于 SKILL.md 的前半部分（ Frontmatter ），由 --- 分割
- YAML 格式，包括且只包括 Name 和 Descrition 两个字段
- Name 与目录名保持一致
- Description 中包括两部分：描述技能做什么 + 具体什么时候用
- ~100 token

## 指令

- 位于 SKILL.md 的后半部分（ body ），由 --- 分割
- Markdown 格式，说明指令
- < 500 行

## 描述

- 位于 agent/openai.yaml
- 在产品列表中显示的名称、简介、图标等信息。它不影响 Agent 行为

**pdf/SKILL.md**

YAML Frontmatter

```
---
name: PDF Processing
description: Comprehensive PDF toolkit for extracting text and
tables, merging/splitting documents, and filling-out forms.
---
```

Markdown

```
## Overview

This guide covers essential PDF processing operations using Python
libraries and command-line tools. For advanced features,
JavaScript libraries, and detailed examples, see ./reference.md.
If you need to fill out a PDF form, read ./forms.md and follow its
instructions.

## Quick Start

```python
from pypdf import PdfReader, PdfWriter

# Read a PDF
reader = PdfReader("document.pdf")
print(f"Pages: {len(reader.pages)}")

# Extract text
text = ""
for page in reader.pages:
    text += page.extract_text()
```
```

**pdf/reference.md**

```
# PDF Processing Advanced Reference

This document contains advanced PDF processing features,
detailed examples, and additional libraries not covered
in the main skill instructions.

## pypdfium2 Library (Apache/BSD License)

### Overview
pypdfium2 is a Python binding for PDFium (Chromium's PDF
library). It's excellent for fast PDF rendering, image
generation, and serves as ...
```

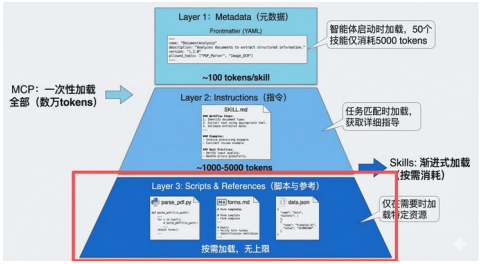
**pdf/forms.md**

```
If you need to fill out a PDF form, first check to see
if the PDF has fillable form fields. Run this script
from this file's directory:
 `python scripts/check_fillable_fields <file.pdf>`,
and depending on the result go to either the "Fillable
fields" or "Non-fillable fields" and follow those
instructions.

# Fillable fields
If the PDF has fillable form fields:
- Run this script from this file's directory:
`python scripts/extract_form_field_info.py <input.pdf>
<fields.json>`.
...
```

**元信息 - 指令示例**

# Skills - 附加资源




## references/

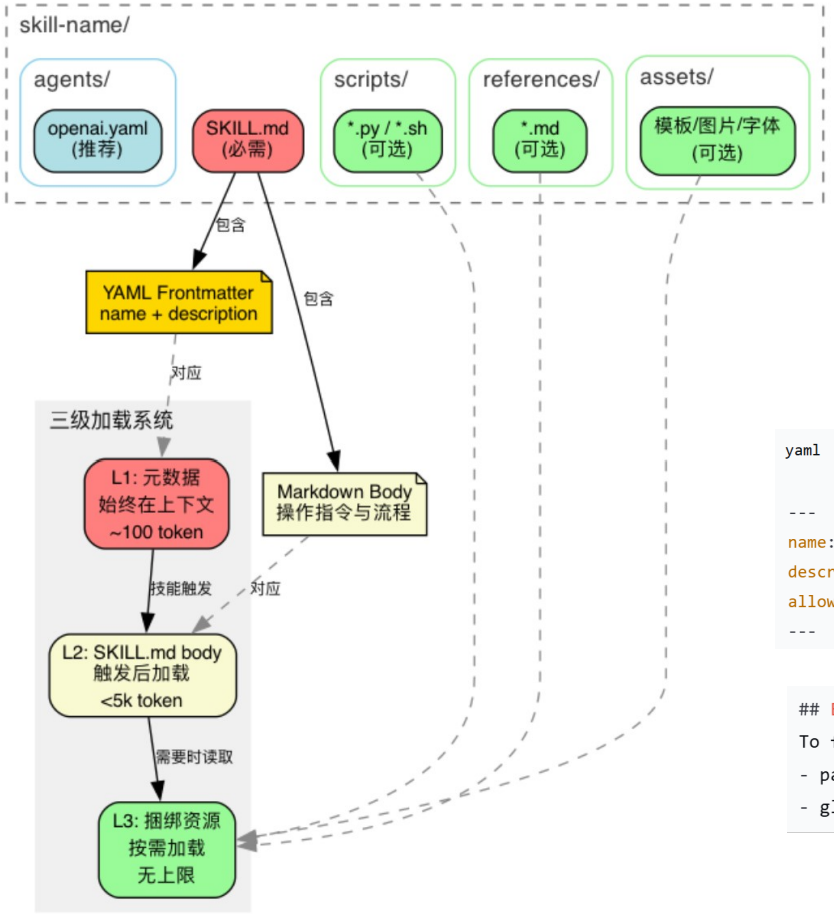
- Agent 在工作过程中需要查阅的参考资料
- 文件链接由 SKILL.md 直接引用，避免 references 互相引用
- 避免 SKILL.md 和 references 内容重复
- 如果 reference 文件 >10K ，看可以在 SKILL.md 设置 grep

## Scripts/

- 可通过 Bash 工具调用的脚本，每个脚本可独立运行
- Agent 需要执行脚本，不需要理解

## assets/

- 直接用于产出的内容， logo ， html 模板等
- Agent 只需要知道把这些资产放到哪里




三级架构加载示意

```yaml
---
name: my-search-skill
description: Search for specific patterns in codebase.
allowed-tools: Read, Grep, Glob
---
```

```
## Examples
To find all Python files that define a `main` function:
- pattern: "def main"
- glob: "*.py"
```

Grep 模式

# Skills - 生成与更新

## Skills 自动化

- 官方 Skill Skill Creator 创建新 Skill 、更新已有 Skill 、评估 Skill 性能
- 从对话中捕捉创建 Skill 意图，提取 Skill 相关信息，创建 Skill 基础版本
- 自动验证 Skill 是否可以被正确触发，并优化技能描述

## Skills Learning in OpenClaw

- 通过 Skill Workshop 进行 Skill 统一管理
- 实施修复和经验总结两条路径更新和管理 Skill Workshop
- 支持周期性改进、合并和移除
- 支持版本管理、虚线回滚




**OpenClaw Skill Self-Learning**

# Skills vs MCP

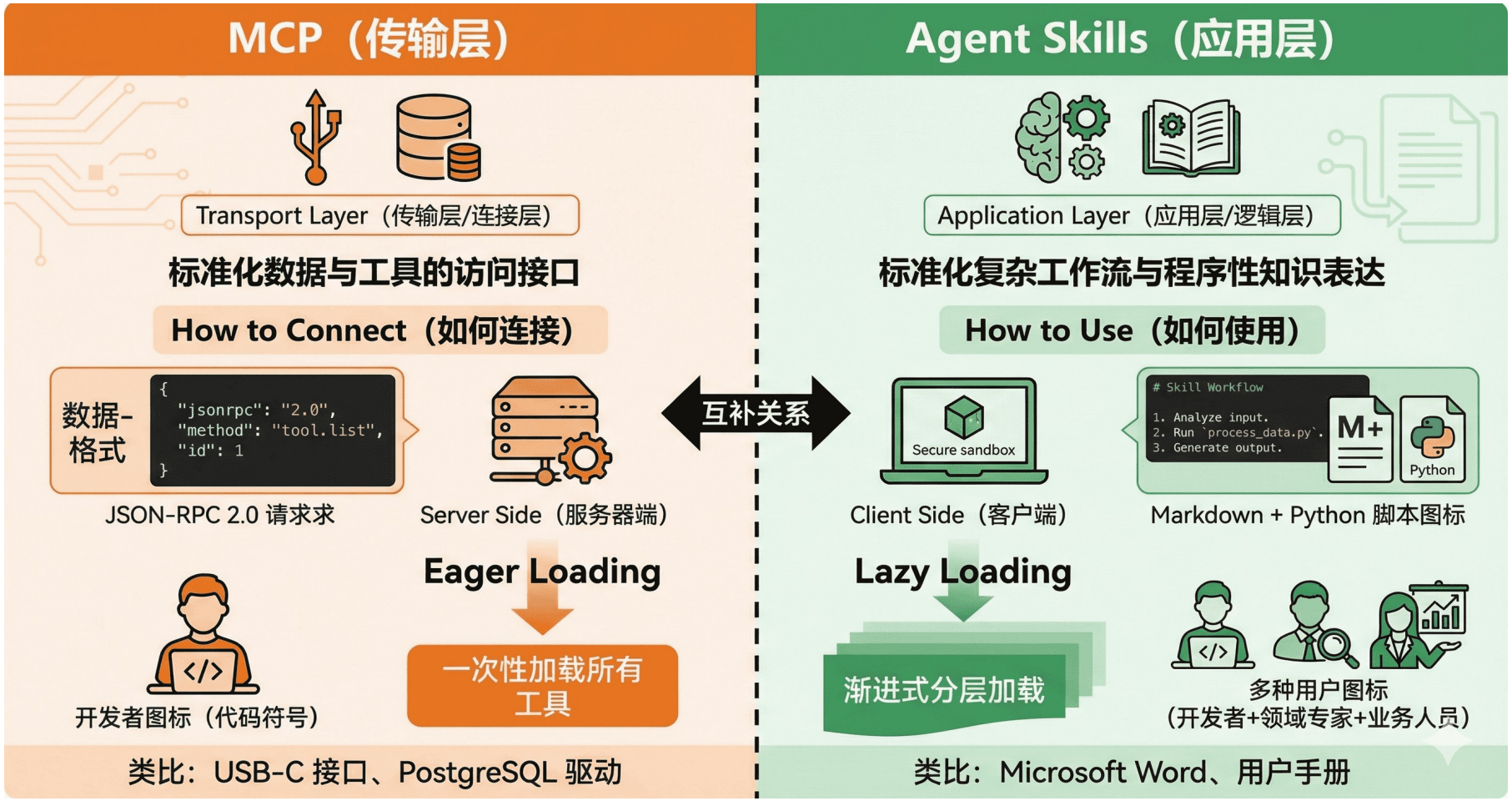
## MCP

- 解决连接性（Connectivity）问题
- 位于传输层
- 提供标准化的访问接口

## Skills

- 解决能力定义（Capability）问题
- 位于应用层
- 提供领域专业知识

**职责分离、架构清晰**




**Skill 和 MCP 比较**

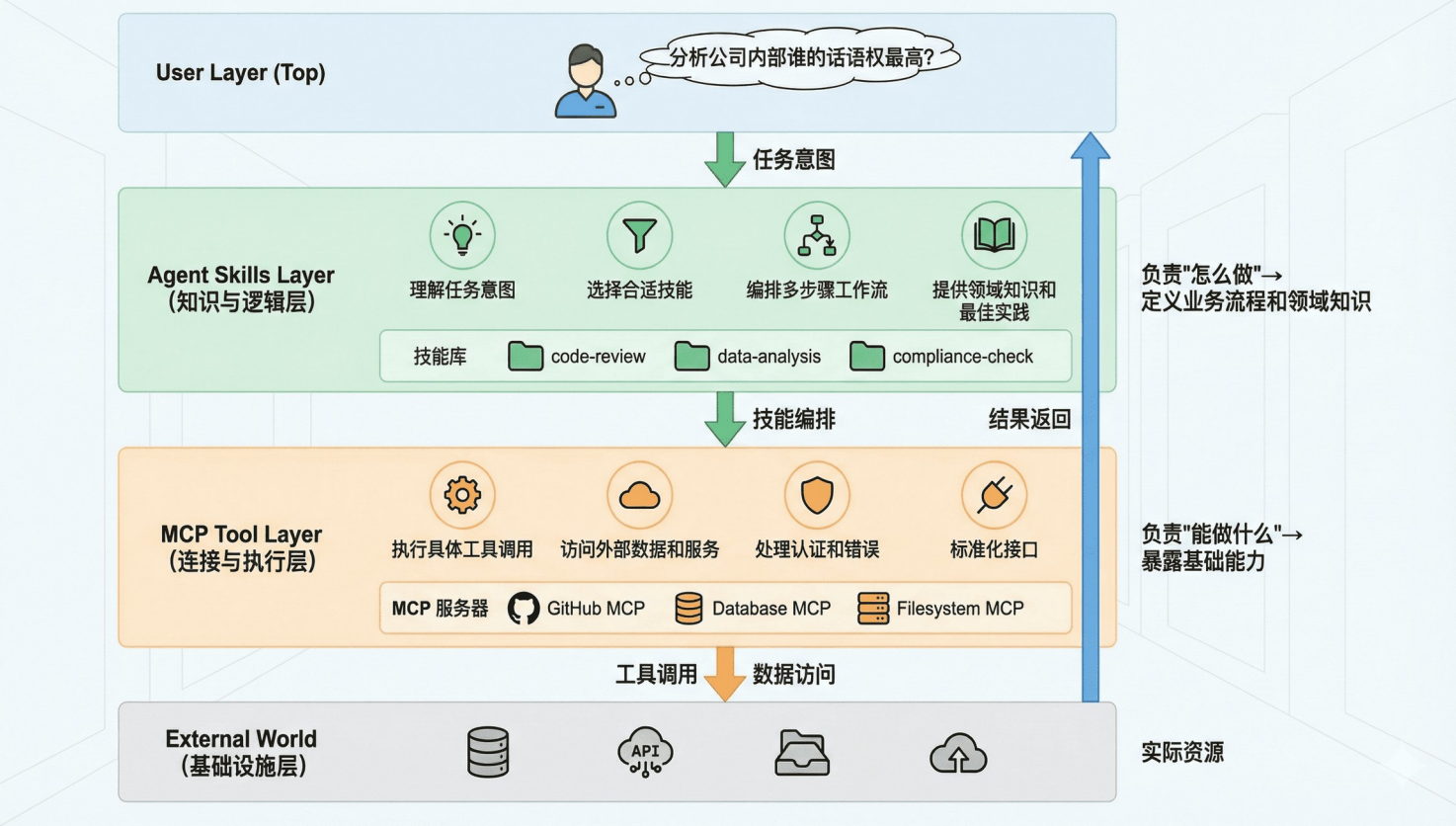
# Skills + MCP

## 典型工作流

- 任务：＂分析团队内部谁的话语权最高＂
- Skills 层识别这是一个数据分析任务，加载对应技能 Skill
- 根据技能指令，将任务分解为子步骤：查询管理关系、薪资对比、任职时长等
- MCP 层执行具体的 SQL 查询，返回结果
- Skills 层根据技能中的领域知识，解读数据并生成综合分析
- 返回结构化的答案给用户

## 架构优势

- 关注点分离： MCP 专注于＂连接＂， Skills 专注于＂知识＂
- 成本优化：渐进式加载大幅降低 token 消耗
- 可维护性：业务逻辑（Skills）与基础设施（MCP）解耦
- 复用性：同一个 MCP 服务器可以被多个 Skills 使用




**Skills + MCP 框架**

# Skills 生态

| Platform | Link | Focus |
|---|---|---|
| Anthropics | https://github.com/anthropics/skills | Anthropic's implementation of skills for Claude |
| OpenAI | https://github.com/openai/skills | Openai Skills Catalog for Codex |
| SkillNet | https://skillnet.openkg.cn/ | Large-scale skill repository and organization |
| ClawHub | https://clawhub.ai/ | Agent skill sharing and discovery |
| SkillHub | https://www.skillhub.club/ | Community skill resources |
| SkillsMP | https://skillsmp.com/ | Marketplace-style skill ecosystem |
| Skills.sh | https://skills.sh/ | Agent skill publishing and reuse |
| BrowserAct Skills | https://github.com/browser-act/skills | Browser automation skills for AI agents with local Chrome reuse, isolated sessions, and human handoff |
| UIZZE | https://uizze.com | Free anti-UI-slop skill and 800,000+ real web and iOS product screens for Codex, Claude Code, Cursor, and other coding agents |
| Hermes Tweet | https://github.com/Xquik-dev/hermes-tweet | Hermes Agent X/Twitter plugin with bundled skill metadata and safe-default social workflow execution |
| Markstream Skills | https://github.com/Simon-He95/markstream-vue/tree/main/.agents/skills | Framework-specific skills for streaming Markdown across Vue, React, Svelte, Angular, Nuxt, Next.js, and Vue 2 |
| ax | https://github.com/Necmttn/ax | Local telemetry graph for coding-agent sessions, skills, tools, and workflow recall |
| Tree Ring Memory | https://github.com/TerminallyLazy/Tree-Ring-Memory | Local-first memory framework for coding agents with lifecycle-aware recall, evidence-backed promotion, forgetting, and a portable Agent Skill package |
| UnifAPI Skills | https://github.com/unifapi-agent/skills | Public-data MCP and KOL pricing skill package for Codex and Claude-compatible agent workflows |
| RunAPI CLI Skill | https://github.com/runapi-ai/cli-skill | Agent workflow skill for running RunAPI image, video, music/audio, and LLM model jobs from Codex and Claude-compatible agents |
| Orkas VideoStudio | https://github.com/Orkas-AI/Orkas-VideoStudio | Source-installable video skills and MCP tools for Codex and Claude Code with editable timelines |
| Before You Build Skill | https://github.com/bin1874/before-you-build-skill | Pre-build product and feature risk review skill for AI coding agents |

# CLI

以命令行方式执行工具调用

## 使用方式

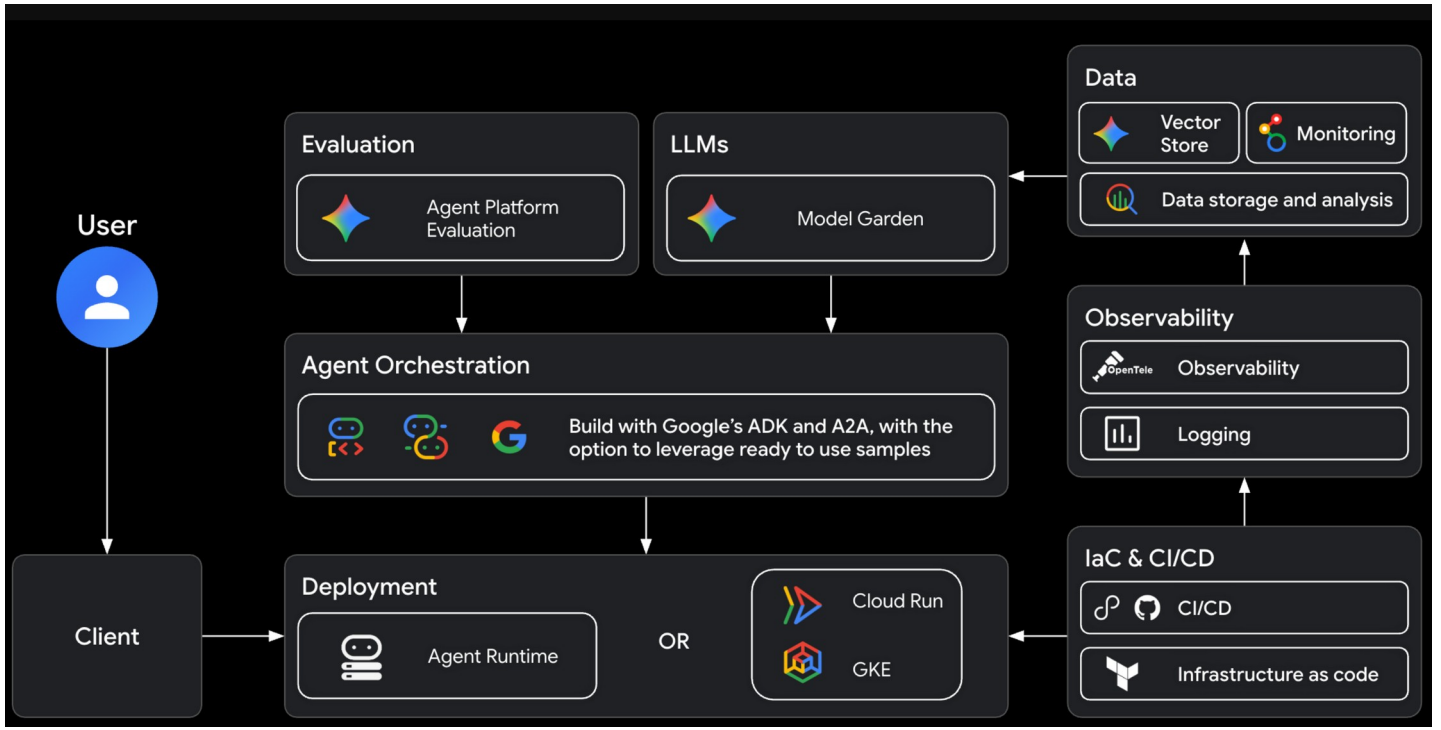
- 面向开发者：快速开发和构建 Agent，替代 IDE
- 面向终端用户：直接使用 Tool 或者 Agent，代替 GUI

## CLI 优势

- 文本输入、文本输出，天然适配 LLM
- CLI 比 GUI 自动化更稳定
- CLI 比直接调 API 更接近 Agent 的工作流
- CLI 比 MCP 包装器更轻量

## CLI 生态

- CLI-Anything、 OpenCLI
- Git 、 GoogleCloud 、 Stripe 、飞书、钉钉、企微




开发者模式 CLI

# SandBox

## 典型场景

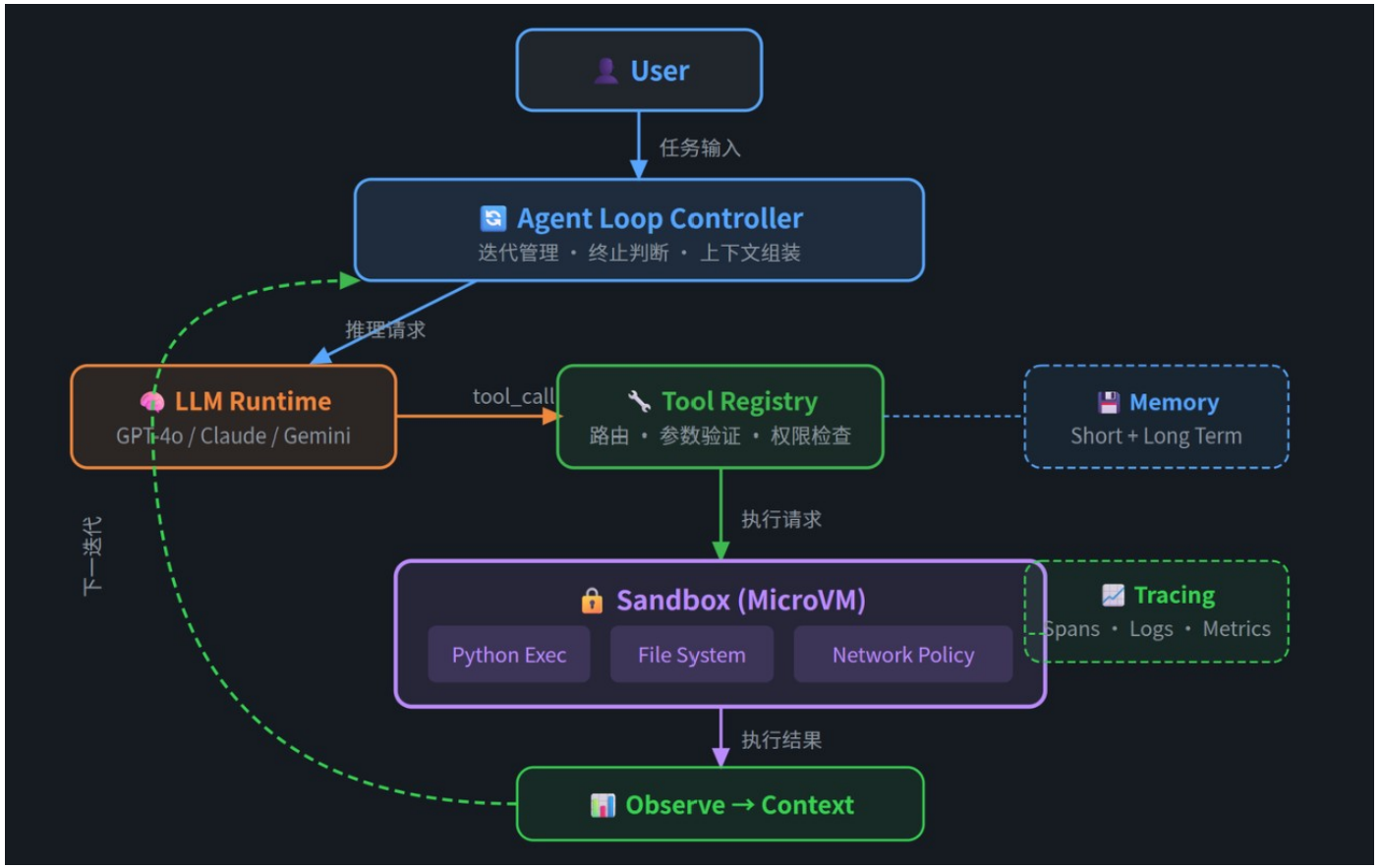
- 代码执行环境
- 可视化操作环境
- ”AI 辅助开发” -> ” 一键部署”

## Agent 沙箱

- 适应未知、动态行为
- 支持长周期的会话
- 默认支持 Agent 技术栈

## 开源实现

- 平台级， K8s 官方实现 agent-sandbox
- 本地轻量级， microsandbox
- 专用沙箱， Skillbox 、 quixand




**Loop + RunTime + Sandbox**

# ltool benchmark

主流评测全景 —— 按「评测维度」划分六大类：从单次调用正确性到真实环境 Agent 行为

## 调用有效性

函数签名与参数是否正确

- BFCL · 行业最主流榜单
- ACEBench · 三档难度覆盖
- T-Eval · 逐步拆解评测
- ToolEyes · 错误归因诊断

已扩展多轮与记忆等新场景

## 工具检索与选择

从海量 API 中选对工具

- ToolBench · 16K+ 真实 API
- StableToolBench · 稳定复现
- API-Bank · 规划检索调用
- Gorilla · 应对 API 演化

规模越大，越考验泛化选择

## 调用决策

该不该调用本身是能力

- When2Call · 需要才调用
- WTU-Eval · 避免多余调用
- BFCL · Miss-Func 场景

拒绝调用同样被计入得分

## 组合与嵌套调用

多步 · 链式 · 嵌套编排

- ComplexFuncBench · 多步约束
- NESTFUL · 嵌套 API 调用
- xLAM · 数据集与评测

一次调用输出喂给下一次

## 多轮真实场景

策略 · 状态 · 意图漂移

- τ-Bench · 策略合规对话
- $\tau^2$-Bench · 更新版评测
- HammerBench · 移动助手
- TRAJECT-Bench · 逐步标注

贴近真实产品的多轮交互

## MCP 专项

标准化协议交互评测

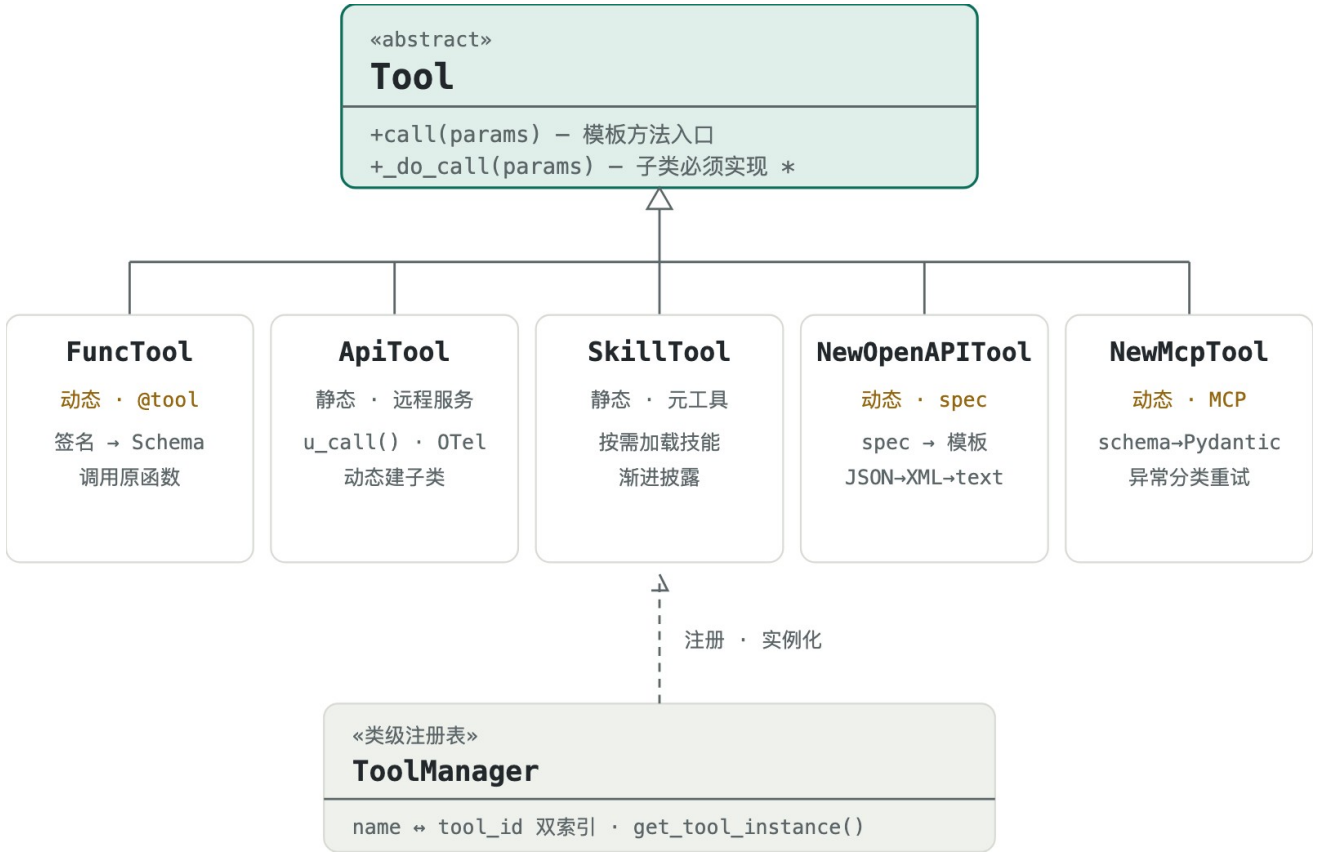
- LiveMCPBench · 多服务器
- MCP-Universe · 真实环境
- MCP-AgentBench · 188 工具

协议互操作成为新战场

**端到端延伸** GAIA · AgentBench · Toolathlon —— 评测完整 Agent 行为（规划 / 记忆 / 环境交互），而非单次调用

# TongAgents 工具系统架构

## 五种工具来源，归一为一个抽象 —— Agent 侧零改动




**1 统一抽象**

无论来源是函数、远端配置、 spec 还是 MCP server，进注册表之前都已经被塑造成同一个 Tool 形状—— Agent 侧完全无感。

**2 一条横切链 · 能力全继承**

校验、沙箱、插件、重试全收在 call() 模板方法——第 6 种工具进来，安全与可观测自动获得

**3 扩展靠喂数据 · 零改存量**

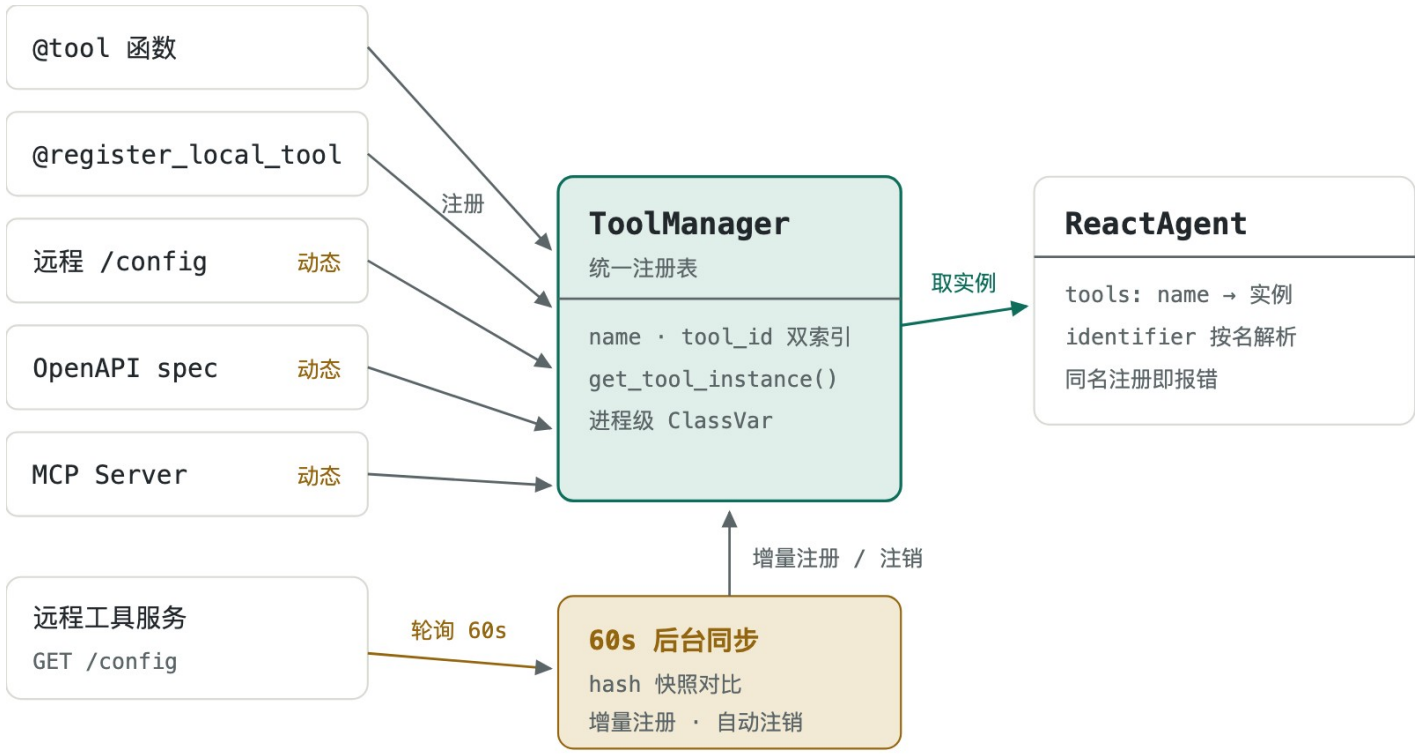
签名、 spec 、 schema 都能运行时建类——接新生态是喂数据，不是改框架代码

| 实现 | 是什么 |
|---|---|
| FuncTool ◆ | @tool 函数 · 签名转 schema |
| ApiTool | 院内部远程服务通道 · u_call / OTel |
| SkillTool | 技能元工具 · 按需加载 |
| NewOpenAPITool ◆ | spec 批量转 REST 工具 |
| NewMcpTool ◆ | MCP server 外部生态接入 |

tongagents/tools/base.py:237-408 · tool_manager.py:42-157 · mcp_tool_manager.py:88 · build_tool_class.py:43

# TongAgents 工具系统架构 - 注册表

## 一张活的注册表 —— 五种来源




1 **ToolManager 双索引**
本地按名、远程按 tool_id 都解析到这张表；Agent 构造时解析一次、持有实例快照，调用热路径零查表

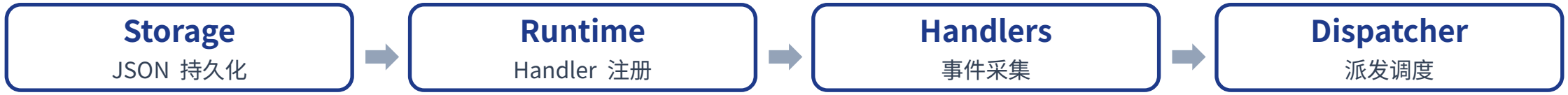
2 **内部 uv 工具增量同步**
60s 轮询对比快照：变化的增量注册、消失的自动注销——增删工具，进程不重启

# TongAgents EventSource 机制

一个轻量级、模块化的事件源调度框架：把 IM/ 定时器 / WebSocket / Webhook 等异构触发源统一为标准事件流，并自动分发给 Agent




四层架构：持久化配置 → 运行时注册 → Handler 采集 → 事件分发。
Handler 事件统一落入 EventStore 并发布到 EventDispatcher 。




### 1 启动链路

- CLI: event-source start
- Runtime: 查 Handler 注册表
- Storage: 加载配置 → 校验
- Handler: handler.start()
- Status: created → running

### 2 事件采集

- lark-cli: subprocess + stdout
- timer: Threading + interval/cron
- bounty-im: WebSocket 客户端
- webhooks: 本地 HTTP server
- 统一调 record_event()

### 3 分发与持久化

- EventStore: 写 events.json
- EventDispatcher: SDK 总线
- SubagentPersister: CLI 镜像
- 下游 : Agent / TUI / Monitor

# 典型应用场景

所有 Handler 通过统一接口接入，可根据业务需要扩展新的事件源。
下面展示 4 种典型应用场景及其 CLI 配置示例。

## Lark IM • lark_cli handler

监听飞书群消息，自动回复或调度 Agent 处理

```
tongagents event-source add lark-cli --webhook https://...
```

## 定时器 • timer handler

定时触发任务（cron / interval），适合周期汇报 / 监控 / 巡检

```
tongagents event-source add timer --schedule "*/5 * * * *"
```

## WebSocket • bounty_im handler

实时双向通信，适合即时通讯 / 长连接场景

```
tongagents event-source add bounty-im --ws-url wss://...
```

## Webhook • webhooks handler

接收 HTTP POST 事件，适合 CI/CD / GitLab / 业务系统集成

```
tongagents event-source add webhooks --path /hook
```

**插件化扩展：tongagents event-source add <type> 即可注册新事件源（HTTP / gRPC / 业务系统）**

# 课后习题

**作业目标**

仿照 cli sample 自己重写一个 cli 客户端，通过本作业掌握 CLI 应用的基本组成： REPL 交互、 React 循环、模型调用、工具注册与工具结果回填，并最终得到一个可以在本地运行、能够真正完成文件操作的 CLI Agent 。

**核心任务**

参考 cli sample 的结构，完成 react loop ， 工具注册， 6 个本地工具等功能

**关键要求**

- 参考 `cli-sample` 的组织方式，但自行完成代码重写；不得用硬编码答案或伪造工具调用代替真实流程。

**最终效果**

**运行客户端后，用户可以在终端中自然语言交互， Agent 会按需选择并执行工具，完成任务后再给出最终回答：**

```text
============================================================
  My CLI Agent | 6 local tools
============================================================
>>> 列出当前目录下的 Python 文件
[调用 glob]
找到: cli_client.py、tools/read_file_tool.py、...
```